DS 1 Informatique - 18 septembre : 4h

Routage dans un réseau arborescent

Dans ce problème, on aborde une question soulevée dans la conception des réseaux : il s'agit d'affecter des adresses distinctes aux nœuds d'un réseau telle que la route entre deux nœuds puisse être déterminée par un calcul utilisant uniquement leurs deux adresses, sans référence à une connaissance globale du réseau. On s'intéresse plus particulièrement aux réseaux ayant une forme d'arbre.

Le problème est composé de trois parties différentes.

Dans tout le problème, un *n-arbre* est un arbre dont l'ensemble des *sommets* (on dit aussi parfois les nœuds) est $[\![0, n-1]\!]$, et dont la racine est 0. Chaque sommet a possède un *père* noté *pere[a]*. Par convention, la racine est son propre père ainsi *pere[0]=0*. L'ensemble des n-arbres est noté $\mathscr{A}(n)$.

L'ensemble des *ancêtres* du sommet a est constitué de a et des ancêtres de son père *pere[a]* ; la racine 0 est ainsi ancêtre de tous les sommets de l'arbre. Le *sous-arbre de racine a* noté A_a, est formé de tous les sommets de l'arbre A dont a est l'ancêtre.

Les *fils* d'un sommet a sont ainsi les $b \neq 0$ tels que *pere[b]=a*, (attention la racine n'est pas fils d'elle même). Une *feuille* est un sommet qui n'a pas de fils.

La *hauteur* (on dit aussi parfois profondeur) d'un sommet a notée $h(a)$ est un entier égale à 0 si a est la racine, elle est égale à la hauteur du père de a augmentée de 1 sinon. Le *plus proche ancêtre commun* à deux sommets a et b, noté *ppac(a,b)*, est le sommet de hauteur maximale qui est à la fois ancêtre de a et de b. Noter que si a est ancêtre de b alors *ppac(a,b)=a*.

Dans tout le problème, on considère que l'entier n est fixé, d'autre part pour un tableau `t`, `t[i]` dénoté l'indice d'élément i ; une liste contenant les entiers $x_0, x_1, \ldots, x_p$ est représentée par $\langle x_0, x_1, \ldots, x_p \rangle$; la liste vide est notée $\langle\rangle$.

On utilisera pour un n-arbre A :
- Un tableau de listes d'entiers `fils` de taille n tels que `fils[i]` est la liste des fils de i. On utilisera le type `int list array` pour cela.
- Un tableau d'entiers `pere` de taille n tel que `pere[i]` est le père de i. On utilisera le type `int array`.

1 Fonctions élémentaires

Question 1 : Calculer *ppac(i,j)* pour tous les couples de sommets de l'arbre A donné plus bas. On présentera le résultat sous forme d'un tableau de 8 lignes et 8 colonnes tel que la case sur la ligne i et la colonne j contienne le plus proche ancêtre de i et j.

Question 2 : Écrire une fonction `hauteur : int array -> int -> int` qui, étant donné le tableau des pères d'un arbre et un sommet a, calcule la hauteur de ce sommet.

Question 3 : Écrire une fonction `filsEnPere : int list array -> int array` qui, à partir d'un arbre donné par ses listes de fils, calcule le tableau des pères correspondant.

Question 4 : Écrire une fonction `ppacMemeH : int array -> int -> int -> int` qui calcule le plus proche ancêtre commun de deux sommets a et b supposés de même hauteur dans un arbre pour lequel on donne le tableau des pères.

Question 5 : Justifier soigneusement la définition de la fonction `ppacMemeH` et analyser la complexité de celle-ci en fonction de $h(a)$.

Question 6 : En déduire une fonction `ppac : int array -> int -> int -> int` qui effectue le même calcul pour deux sommets a et b n'ayant pas nécessairement la même hauteur.

2 Arbres binaires complets

Un *arbre binaire complet* est un arbre dans lequel tout sommet qui n'est pas une feuille a exactement deux fils, et tel que toutes les feuilles sont à la même hauteur. On note $\mathscr{B}(h)$ l'ensemble des n-arbres binaires complets dans lesquels les feuilles sont à hauteur h. Un exemple d'arbre binaire complet B est donné à droite de la figure 1.

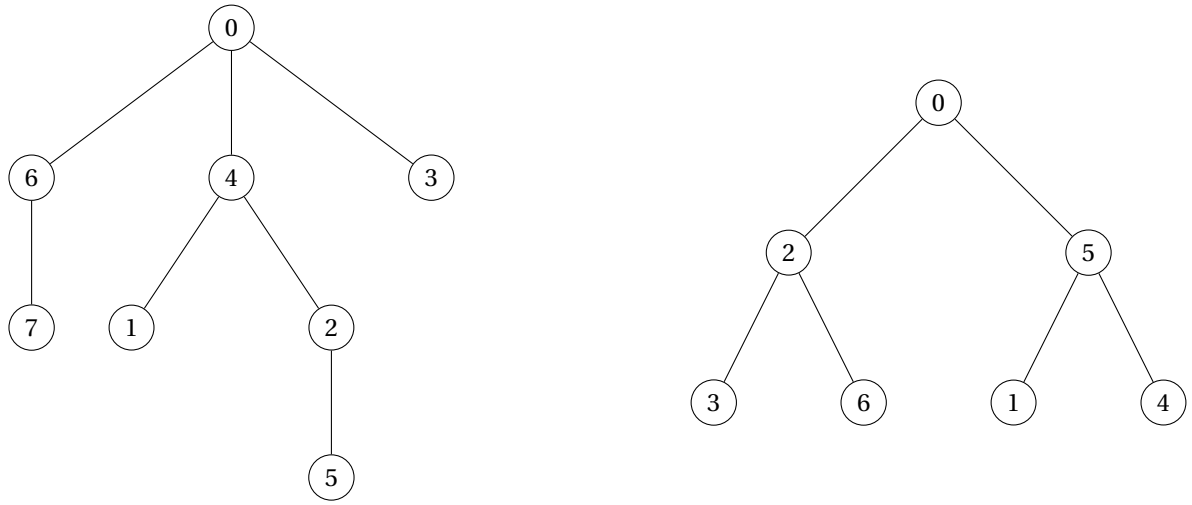

FIGURE 1 – Un arbre $A \in \mathscr{A}(8)$ et un arbre binaire complet $B \in \mathscr{B}(2)$.

Question 7 : Déterminer pour $0 \leq k \leq h$ le nombre s_k de sommets de hauteur k dans un arbre B de $\mathscr{B}(h)$, justifiez votre réponse. En déduire le nombre n de sommets d'un arbre de $\mathscr{B}(h)$ en fonction de h.

Dans la suite, B est un arbre de $\mathscr{B}(h)$ ayant n sommets; à chaque sommet a de B, on associe une étiquette $l(a) \in [\![1, n]\!]$ satisfaisant la condition suivante :
Pour chaque sommet a ayant pour liste de fils $\langle b, c \rangle$:

$$\max_{u \in B_b} l(u) < l(a) < \min_{v \in B_c} l(v)$$

Question 8 : Calculer les valeurs de $l(a)$ pour les 7 sommets de l'arbre binaire complet représenté sur la droite de la figure 1 : l'ordre des fils dans les listes correspond à la lecture de gauche à droite de la figure.

Question 9 : Démontrer l'unicité et l'existence de la fonction d'étiquetage pour tout arbre.
Question 10 : En déduire une fonction `etiquettes : int list array -> int array` qui calcule les étiquettes $l(a)$ pour tous les sommets d'un arbre $\mathscr{B}(h)$ donné par ses listes de fils.

Question 11 : Un arbre B de $\mathscr{B}(h)$ de racine 0 ayant pour liste de fils $\langle a, b \rangle$ se décompose en deux sous-arbres B_a et B_b. Déterminer les valeurs de $l(0)$, de $l(a)$ et de $l(b)$ pour les fils a et b de la racine.

Question 12 : Démontrer que si a et b sont deux sommets quelconques de B, alors *r=l(ppac(a,b))* est déterminé de manière unique à partir de $p = l(a)$ et $q = l(b)$ par la propriété suivante : r est parmi les entiers compris (au sens large) entre p et q celui possédant le plus grand diviseur qui soit une puissance de 2.

Question 13 : Écrire une fonction récursive `mu : int -> int -> int` qui, détermine *r=l(ppac(a,b))* à partir de $p = l(a)$ et $q = l(b)$; on supposera que $p \leq q$. La fonction doit être de complexité $O(\log_2(q-p))$.

3 Arbres généraux

Dans cette partie, on utilise les définitions et notations suivantes :

Dans un arbre A, le poids $w[a]$ d'un sommet a est le nombre de sommets du sous arbre A_a de racine a, ainsi $w[a] = 1$ si et seulement si a est une feuille, noter que $w[0] = n$ (c'est le nombre de sommets de A).

Un arbre A est dit *gauche* si pour chaque sommet a (qui n'est pas une feuille) le premier élément de la liste est de poids supérieur ou égal à celui de tous les autres sommets de cette liste. Dans un arbre gauche, le premier élément de la liste des fils d'un sommet est dit *lourd*, les autres sommets sont dits *légers*. La racine est un sommet léger.

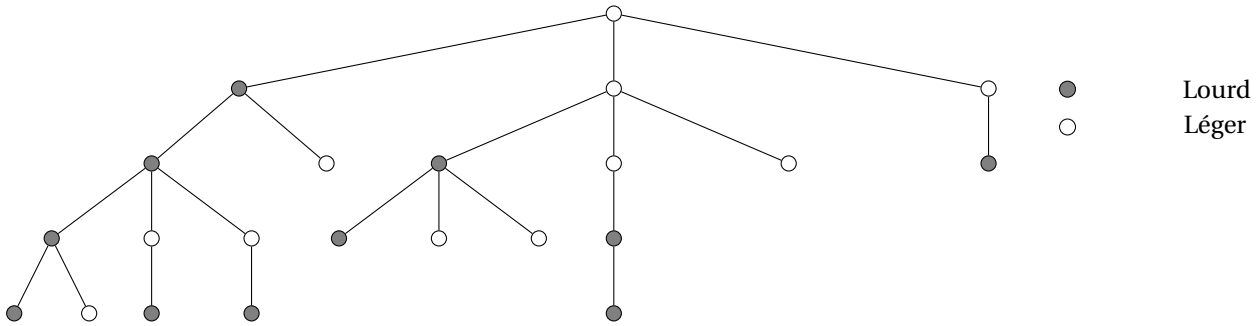

FIGURE 2 – Sommets lourds et légers

Question 14 : Écrire une fonction `poids : int list array -> int array` qui calcule les poids de tous les sommets donné par ses listes de fils.

Question 15 : Écrire une fonction `gauchir : int list array -> int array -> int list array` qui calcule un arbre gauche obtenu en réordonnant les fils de chaque sommet de façon telle que le premier fils soit de poids supérieur ou égal à celui des autres. Sont donnés dans cette fonction les listes des fils et le tableau des poids des sommets de l'arbre.

Question 16 : Soit a un sommet léger d'un arbre gauche A et b son père, montrer que $w[b] > 2w[a]$. En déduire que le nombre d'ancêtres légers de a est inférieur à $1 + \log_2 n$.

On utilise dans toute la suite la notion de *cime* d'un sommet.

- Si a est léger, *cime[a]* est égale à *pere[a]*.
- Si a est lourd, *cime[a]* est le plus proche ancêtre de a qui est léger.

Ainsi, en raison de nos conventions, *cime[0]=0*. On appelle A' l'arbre gauche obtenu en appliquant la fonction `gauchir` à l'arbre A de la figure 1. Les cimes des sommets de l'arbre A' sont alors donnés par le tableau suivant :

i	0	1	2	3	4	5	6	7
cime	0	4	0	0	0	0	0	6

Question 17 : Écrire une fonction `cimes : int list array -> int array` qui calcule les sommets cimes de tous les sommets à partir des listes de fils d'un arbre gauche.

On se propose d'attribuer des étiquettes aux sommets d'un arbre gauche A. Pour cela, on commence par construire deux tableaux d'entiers *t1* et *t2* associés aux sommets.

Le tableau $t1$ est tel que *t1[0]=0* et *t1[a]=i* si a est le i-ème élément de la liste des fils de son père.

Le tableau $t2$ est tel que *t2[a]=0* si a est léger et *t2[a]=t2[pere[a]]+1* si a est lourd.

Les étiquettes $\lambda(a)$ des sommets sont alors des listes d'entiers construites de la façon suivante :

$$\lambda(0) = \langle\rangle \qquad \text{et pour} \quad a \neq 0 \;\; \lambda(a) = \lambda(cime[a]) \bullet \langle \mathrm{t1[a], t2[a]} \rangle$$

dans cette formule, $\bullet$ dénote la concaténation des listes.

Question 18 : Donner l'arbre A' défini entre les questions 16 et 17 ; puis donner les valeurs *t2[a]* pour tous les sommets a ; calculer enfin leurs étiquettes $\lambda(a)$.

Question 19 : Écrire une fonction `etiquettes : int list array -> int array -> int list array` qui calcule les étiquettes des sommets d'un arbre gauche donné par ses listes de fils et pour lequel on donne aussi le tableau des cimes des sommets. On pourra utiliser l'opérateur `@` qui calcule la concaténation de deux listes.

Question 20 : Écrire une fonction `trouve : int list array -> int list -> int` qui, à partir de l'étiquette

d'un sommet d'un arbre gauche A et des listes de fils des sommets de cet arbre, détermine ce sommet. On justifiera soigneusement sa correction.

Question 21 : Montrer que, pour tout sommet a d'un arbre, la longueur de l'étiquette $\lambda(a)$ est majorée par 4 fois le nombre de ses ancêtres légers.

Question 22 : Expliquer comment calculer $\lambda(ppac(a,b))$ à partir des deux listes $u = \lambda(a)$ et $v = \lambda(b)$.

Question 23 : Écrire une fonction `ppac2 : int list array -> int list array -> int -> int -> int` qui, étant donné les listes de fils des sommets d'un arbre gauche, le tableau des étiquettes des sommets ainsi que deux sommets a et b, calcule *ppac*(a,b).

Question 24 : Analyser la complexité de la fonction `ppac2`.